SLIDE 01 — DARREN

Enterprise expense reimbursement is *broken at scale.*

Finance teams are processing hundreds of claims a month — manually, inconsistently, without a structured audit trail.

01
Receipt submitted
Free-text, PDF, or both — unstructured

→ 02
2 email handoffs
Manual routing across teams

→ 03
Policy lookup
Institutional memory, inconsistently applied

→ 04
Duplicate search
Manual, error-prone, frequently missed

→ 05
Manual data entry
Before any decision is made

→
Several business days
No audit trail.
No machine-readable record.

SLIDE 02 — DARREN

Existing tools validate forms. They don't *exercise judgment.*

WHAT ERP TOOLS DO

Basic form validation

- Check that fields aren't empty
- Verify a number is a number
- Route to a predefined approver

WHAT THEY CAN'T DO

Contextual reasoning

- Flag a MYR 500 receipt on a MYR 50 claim
- Detect a duplicate Notion licence when free seats exist
- Distinguish ambiguous claims from outright violations

SLIDE 03 — DAVID

ORI◆N

ORCHESTRATED REIMBURSEMENT
INTELLIGENCE OPERATIONS NETWORK

Deterministic rule engine for non-negotiable policy.
LLM-powered intelligence for claims requiring judgment.

<30s
End-to-end processing
vs. several business days

- **Consistent enforcement**
  Every policy rule applied identically on every single claim — no institutional memory drift.
- **Contextual reasoning**
  LLM intelligence for fraud signals, duplicate detection, and receipt cross-checking.
- **Complete audit trail**
  Full reasoning traces for every decision — replayable, linked, machine-readable.

60–70%
processing time reduction

100%
autonomous on clear-cut cases

David · ~1 min

SLIDE 04 — DAVID

A stateful graph, *not a linear pipeline.*

ORCHESTRATION

LangGraph DAG

Directed acyclic graph — stateful nodes accumulate signals into a central WorkflowState. Routes each claim based on what it finds, not a fixed sequence.

HYBRID EXECUTION

Deterministic + LLM

Policy checks run in pure Python — 100% consistent, no LLM cost. Intelligence node handles fraud signals, receipt cross-checking, routing decisions on ambiguous cases.

STACK

FastAPI · ILMU GLM-5.1 · pydantic v2

OpenAI-compatible LLM interface. Strict schema validation at every state transition. LangSmith for full observability — every agent step linked into one trace tree per submission.

40%

latency reduction from running Intelligence and Policy nodes in parallel

100%

consistency on hard rules — deterministic Python, not LLM

PARALLEL BRANCH

Intelligence || Policy nodes branch concurrently — merge results before Supervisor decision.

Five possible outcomes. *One adaptive path.*

INPUT
Claim submitted

→

AGENT 01
Intake Agent

→

POLICY NODE
7 rule checks

INTELLIGENCE NODE
Fraud · Duplicates · Receipts

→

MERGE + ROUTE
Supervisor

FIVE POSSIBLE OUTCOMES

01 Auto-approve	02 Auto-reject	03 Request clarification	04 Escalate to manager	05 Escalate to finance

ADAPTIVE ROUTING

If Policy issues a hard block and Intake confidence is high, Supervisor skips Validation entirely. The graph doesn't run steps it doesn't need.

SLIDE 06 — VANESSA

Built to handle the cases that *break other systems.*

01

Semantic duplicate detection

"ChatGPT Plus" and "OpenAI GPT" resolve to the same licence. "Notion Labs Inc." and "Notion" are the same vendor. Fuzzy matching plus LLM disambiguation — the level a human reviewer would apply, automated.

02

Alternative recommendations

If a duplicate has free seats available, or a cheaper tier exists, ORION doesn't just reject — it tells the employee what to use instead, with an estimated cost saving in MYR. Actionable, not just a rejection notice.

03

Validation Agent

When a claim is genuinely ambiguous — not a violation, just incomplete — the system generates 1–3 targeted clarifying questions and sends them back to the employee. It doesn't force a low-confidence rejection.

04

Audit trail

Every decision persisted to ledger.json with claim ID, employee ID, amount, decision, timestamp, and a LangSmith deep-link. A reviewer can click that link and replay the entire decision step-by-step.

SLIDE 07 — JASON

Two-tier testing. *Built for a regulated decision system.*

TIER 01 — OFFLINE

Every pull request

- Stub LLM exercises every LangGraph node — no API calls, under 3 seconds
- Catches schema regressions, routing logic errors, coverage drops before main
- 80% line coverage gate enforced via CI

RTM

Every requirement maps to a numbered test. No untested code path. No unplanned feature creep reaching production.

TIER 02 — LIVE

Nightly against real endpoint

- Runs against the real ILMU GLM-5.1 endpoint
- Asserts decisions within hand-curated expected bands — not exact strings
- Prompt drift doesn't break the build. Silent quality regression does.

ADVERSARIAL TEST

Injected instruction to bypass all policy checks → treated as receipt data, flagged by Critic node. Structured output enforcement held.

SLIDE 08 — JASON

Numbers from actual test runs, *not projections.*

95/95
Tests passing

81.49%
Line coverage — 1.49pp above CI gate

11.4s
Happy-path end-to-end response

6.8s
Fast-reject path — Supervisor LLM call skipped

PARALLEL BRANCH

8.2 s

Combined wall-clock for 20 submissions. Intelligence avg 8.1s, Policy 0.3s deterministic. Max, not sum — parallel branch working as designed.

IDEMPOTENCY UNDER LOAD

38 ms

Cache-hit avg. 20 concurrent identical submissions → exactly 1 ledger entry, same claim ID and decision for all 20 callers.

PROCESSING IMPROVEMENT

60–70 %

Estimated reduction in processing time. Finance only touches cases that genuinely need a human.

ORI•N

SLIDE 09 — JASON

Built by four people with *clear ownership.*

THE TEAM

David	Agent topology · LLM service layer · Schema contracts · CI/CD
Vanessa	Per-agent prompts · Temperature config · LangSmith trace analysis
Darren	Demo UI · Workflow diagram · Stakeholder walkthrough
Jason	Test infrastructure · RTM · Coverage gates · Release sign-off

PHASE 2 PRIORITIES

P2.1 **Ledger → SQLite**
Append-only JSON works at low volume. Not safe for concurrent multi-instance writes.

P2.2 **RBAC reviewer dashboard**
Escalated claims currently have no approver UI.

P2.3 **ERP integration**
Automated payroll sync once decision quality is proven in production.

Procurement validation and invoice processing are natural adjacencies on the same graph architecture.